State Bank of India

Operating CRM at 300,000 users — what enterprise scale actually changes

A structural analysis of the largest CRM deployment in Indian banking: where scale stops being a capacity question and becomes an architecture, integration and governance question.

300K+	45 Cr+	23,000+	19	250+	8
platform users	customers	branches	countries	integration touchpoints	lines of business

REFERENCE DEPLOYMENT · VENDOR-PUBLISHED FIGURES

30,000+ concurrent users · 50+ integrated systems · 178 real-time APIs · 13 international go-lives

Aditya Singh · Analysis authored for solution and bid strategy · 2026

Vendor and platform names removed. This is an analysis of a reference deployment, not a claim of delivery ownership — see slide 3.

EXECUTIVE SUMMARY

At 300,000 users the hard problems stop being functional and become structural — integration surface, concurrency, jurisdiction and change throughput

Situation

A single CRM instance serving 300,000+ users and roughly 45 crore customers across 23,000+ branches, eight lines of business and nineteen countries.

Functionally it does what any bank CRM does: leads, activities, service requests, campaigns, dashboards.

Complication

None of the difficulty is in the feature list. It sits in four places that only appear above a certain size: 30,000 concurrent sessions, 250+ integration touchpoints, thirteen jurisdictions with different data rules, and a change queue shared by eight businesses.

Each of these fails in a different way, and none is solved by adding capacity.

The argument

Above a threshold, scale converts three design choices from preferences into constraints: the integration contract, the tenancy and residency model, and who owns the change queue.

Get those wrong and no amount of engineering recovers it — the deployment becomes unchangeable rather than slow.

THE NUMBERS THAT MATTER

300K+	30K+	250+	178	19	8
platform users	concurrent sessions	integration touchpoints	real-time APIs	countries, 13 live	lines of business

Why this analysis matters: most CRM programmes are sized for the user count and then surprised by the integration surface, the jurisdictional spread and the change-throughput ceiling. Those three, not concurrency, are what actually decide whether a deployment of this size stays viable.

Source: Reference-deployment figures published by the platform vendor, retained with attribution

HOW TO READ THIS DECK

This is analysis of a reference deployment, not a delivery claim — the boundary is stated here so nothing in the deck has to be inferred

VENDOR-PUBLISHED

Reference context

Deployment figures published by the platform vendor. Retained with attribution and not independently audited.

- 300,000+ platform users
- 45 crore+ customers
- 23,000+ branches
- 30,000+ user concurrency
- 50+ integrated systems
- 19 countries, 13 go-lives

ANALYSIS

My contribution

The structural argument, the threshold model and the design rules. Authored to support solution and bid strategy on comparable enterprise pursuits.

- The four-threshold framework
- Integration-surface decomposition
- Change-throughput ceiling model
- Jurisdiction and residency analysis
- Design rules at this scale
- Applied to live enterprise bids

ILLUSTRATIVE

Modelled

Curves and thresholds used to make the structural point. Directionally argued from the published figures, not measured.

- Superlinear cost curve
- Change-queue contention model
- Concurrency failure modes
- Rollout sequencing logic
- Replace with programme actuals
- Argument holds either way

OUT OF SCOPE

Not claimed

Explicitly outside this deck. Stated so the boundary is unambiguous rather than left to the reader.

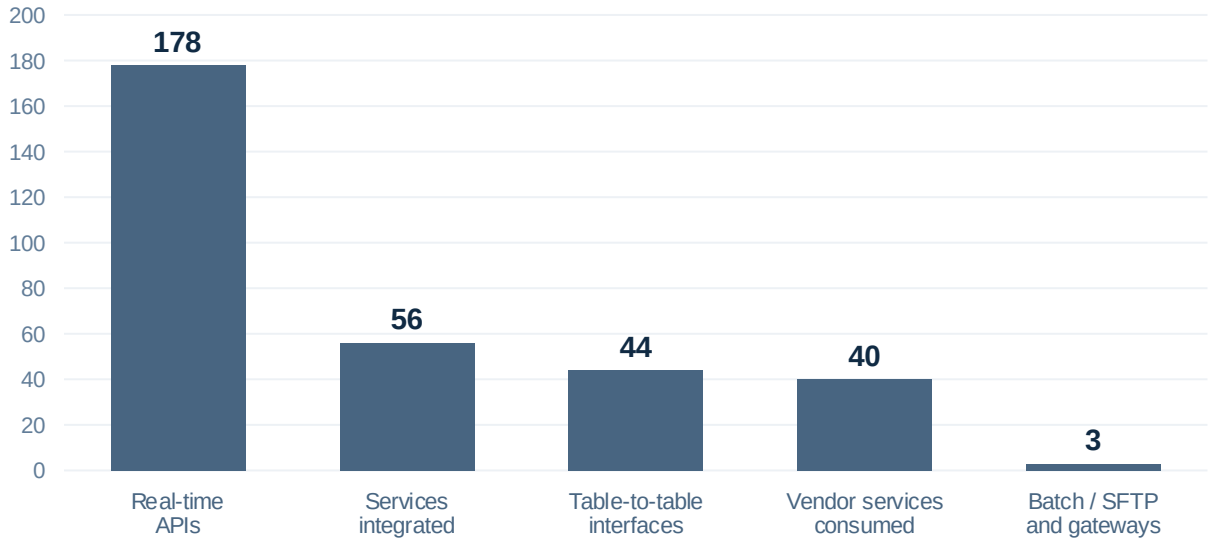
- Delivery ownership of this programme
- Any role on the implementation team
- Access to the bank's internal data
- Any confidential design artefact
- Independently audited outcomes
- Commercial terms of the engagement

Source: Vendor-published reference figures (context) · author's analysis (argument and models)

THE SCALE

Every dimension is an order of magnitude above a typical enterprise CRM — which is why the usual design intuitions stop applying

300K+	30K+	45 Cr+	23,000+	50+	250+	19	13
platform users	concurrent sessions	customers	branches	integrated systems	integration touchpoints	countries in scope	international go-lives



Decomposition of the integration surface — 250+ touchpoints across five interface classes

Why the surface matters more than the size

- User count drives infrastructure cost, which is the cheapest problem on the list to solve.
- The integration surface drives regression scope — every release must prove 250+ contracts still hold.
- 178 real-time interfaces mean the CRM inherits the availability of everything behind it.
- 44 table-to-table interfaces are the ones that fail silently and are discovered by a reconciliation, not an alert.

The number that should worry a programme director is not 300,000 users. It is 178 real-time interfaces, because that is the number that determines how long a release takes to regression-test and how many other systems can take the CRM down.

Source: Reference-deployment figures published by the platform vendor, retained with attribution

THE ARGUMENT

Four thresholds turn design preferences into hard constraints — and each one fails in a completely different way

30,000+

Concurrency

Simultaneous sessions against one logical instance.

HOW IT FAILS

Fails as latency, then as timeouts, then as duplicate records when users retry.

THE DESIGN RESPONSE

Isolate high-volume transaction paths; separate read and write workloads; cap synchronous chains.

250+

Integration surface

Touchpoints across 50+ systems, 178 of them real-time.

HOW IT FAILS

Fails as regression cost — releases slow until change effectively stops.

THE DESIGN RESPONSE

Contract-first interfaces, versioned; no interface without an owner and a test.

19

Jurisdiction

Countries in scope, thirteen live, a subset under data-protection regimes.

HOW IT FAILS

Fails at audit, not in production — and remediation is a re-architecture.

THE DESIGN RESPONSE

Residency and tenancy decided before the first go-live, never retrofitted.

8

Change throughput

Lines of business sharing one change queue and one release train.

HOW IT FAILS

Fails politically — the largest business wins every slot and the rest fork.

THE DESIGN RESPONSE

Allocate release capacity by rule, not by escalation; publish the queue.

Only the first is an engineering problem. The other three are decided by architecture and governance, and all three are irreversible once the first country is live.

Source: Author's analysis of the reference deployment

JURISDICTION

Nineteen countries on one platform means thirteen live regulators — and a data-residency decision that cannot be revisited after the first go-live

South Asia

Nepal
Bangladesh
Sri Lanka
Maldives

Asia Pacific

Singapore
Hong Kong
Shanghai
Japan
Indonesia
Australia

Middle East & Africa

Dubai
Bahrain
South Africa
Mauritius

Europe & Americas

UK Retail
UK Corporate
Frankfurt
USA
Canada

19
countries in scope

13
go-lives completed

6
under data-protection dependency

1
logical platform serving all of them

What multi-jurisdiction actually forces

- A residency decision per country before the first record exists there.
- A tenancy model that can isolate one country without cloning the platform.
- Consent and retention rules that differ by jurisdiction on one data model.

Why it cannot be retrofitted

- Residency is a storage-location property; changing it means migrating live production data under supervision.
- A regulator's first question is where the data has been, not where it is now.
- Six of nineteen carrying a data-protection dependency means the strictest regime sets the design.

The rule this produces

- Design for the strictest jurisdiction in scope, then relax by exception — never the reverse.
- Sequence go-lives so the hardest regulator is early, not last.
- Keep one data model; vary policy, not structure.

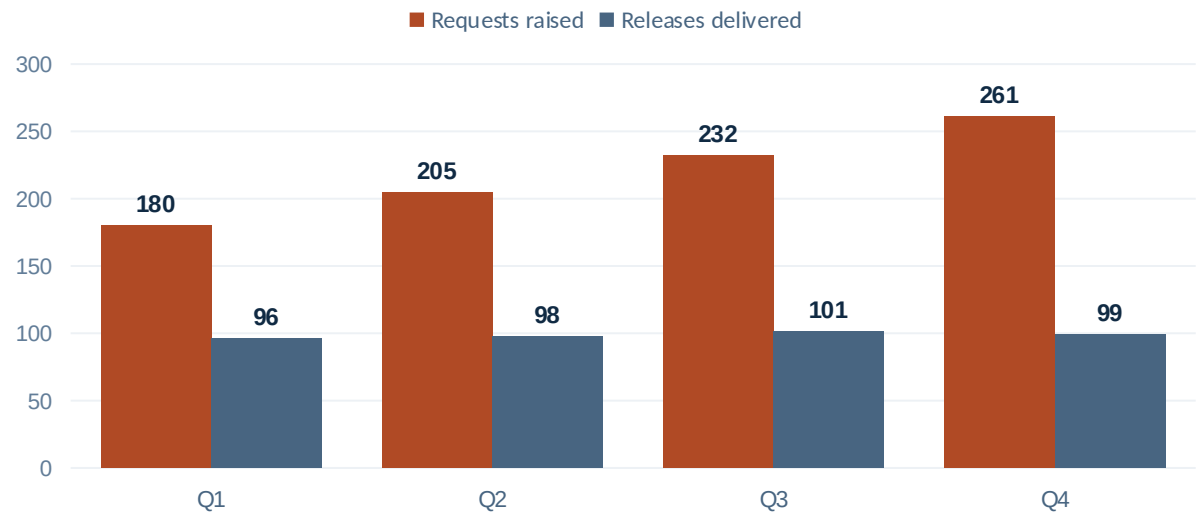
Source: Reference-deployment figures published by the platform vendor, retained with attribution · author's analysis

CHANGE THROUGHPUT

Eight lines of business share one release train — which makes release capacity, not engineering capacity, the scarcest resource in the programme

Personal Banking	Corporate Banking	NRI Banking	Agri Banking	SME Banking	MCS Banking	Global Markets	Government Business

Eight businesses · one logical platform · one change queue · one release train



Illustrative change-queue contention: demand grows with the business count, throughput does not

How the queue actually gets allocated

- Without a rule, capacity goes to whoever escalates highest — usually the largest business.
- The businesses that lose stop asking and start building shadow systems, which reappear later as integration debt.
- Published, rule-based allocation is less efficient per release and far more stable across a year.
- Reserving a fixed share for platform work is the only way the shared spine survives contact with delivery dates.

The governance point

At eight businesses, the release train is a scarce shared resource being allocated by negotiation. Converting that to a published rule is not bureaucracy — it is the mechanism that stops the platform fragmenting into eight forks over three years.

Source: Author's analysis of the reference deployment

FOOTPRINT

The functional scope is unremarkable — which is the point: nothing on this slide explains the difficulty, and that is where programmes misjudge the estimate

Sales automation

- Lead management
- Sales planning
- Activity management
- Field mobility

Relationship mgmt

- Retail relationships
- Corporate relationships
- SME relationships
- Agri relationships

Service automation

- Contact centre
- Complaint management
- Request tracking
- SLA and escalation

Marketing

- Campaign repository
- Campaign execution
- Budget and expense
- Response capture

Reporting

- Pipeline reporting
- Executive dashboards
- SLA tracking
- Performance views

What the feature list predicts

- A standard enterprise CRM build: five module families, familiar objects, well-understood workflows.
- An estimate driven by user count, module count and configuration volume.
- A timeline that looks like every other CRM programme, scaled by headcount.

What actually drives the effort

- Regression across 250+ integration contracts on every release.
- Residency and tenancy work that has no visible feature output at all.
- Reconciling eight businesses onto one object model without forking it.
- Thirteen go-lives, each with its own regulator, cutover and support model.

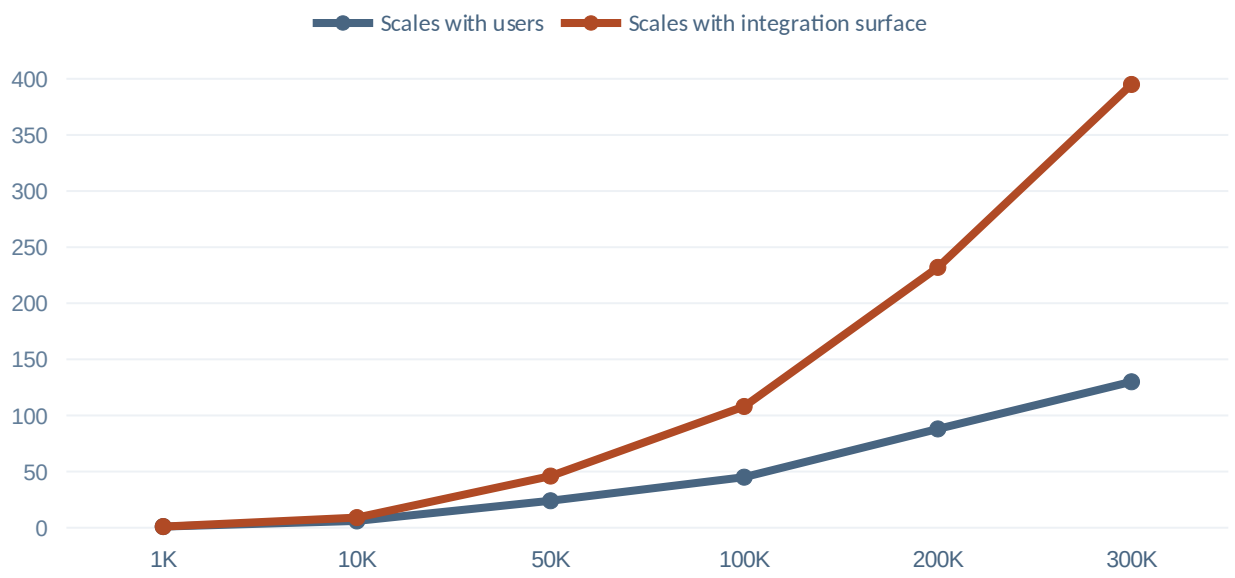
The estimating error this causes

- Sizing by functional scope understates a programme of this shape by a wide margin.
- The invisible work — contracts, residency, governance — is most of the cost and none of the demo.
- Sponsors approve the visible half and are then surprised twice: at go-live, and again at the first major upgrade.

Source: Reference-deployment figures published by the platform vendor, retained with attribution · author's analysis

ECONOMICS

Three cost lines scale linearly with users and three scale with the square of the integration surface — only the first three are in most business cases



Illustrative cost index by platform user count — the shape is the argument, not the values

Linear — usually costed

- Infrastructure and licensing
- Training and rollout
- Level-one support capacity

Superlinear — usually missed

- Regression and assurance — grows with interface count, not user count
- Environment and test-data management across jurisdictions
- Coordination cost across eight businesses and thirteen country teams

Why this is the most common business-case error at scale

A programme costed on the linear lines looks affordable and is approved. The superlinear lines then appear as overruns in year two, get labelled delivery underperformance, and are managed by cutting scope — which raises cost per unit of change and makes the curve worse. The correct response is to reduce the integration surface, not the scope.

Source: Author's cost model — illustrative index, structural argument

DESIGN RULES

Seven rules that are optional below 50,000 users and non-negotiable above it

Contract-first interfaces

Every interface versioned, owned and independently testable. No interface enters production without a named owner and an automated contract test.

One object model, varied policy

Eight businesses and nineteen countries share one structure. Difference is expressed as configuration and policy, never as a schema fork.

Residency decided before record one

Storage location and tenancy fixed before the first go-live in any jurisdiction. Retrofitting means migrating live production data under supervision.

Published release allocation

Change capacity allocated by rule, with a fixed reserve for platform work. Escalation-based allocation guarantees fragmentation within three years.

Synchronous chains capped

No user action may depend on more than a bounded number of real-time hops. Beyond that, the CRM inherits the availability of everything behind it.

Reconciliation as a first-class control

Table-to-table interfaces fail silently. Every one needs a scheduled reconciliation and an owner, not an alert nobody reads.

Hardest regulator first

Sequence go-lives so the strictest jurisdiction is early. Designing for the easiest and generalising later is the most expensive possible order.

Source: Author's analysis of the reference deployment

WHAT TRANSFERS

Four judgements that carry to any platform above this threshold

01 **Size the surface, not the seats**

User count is the cheapest constraint to solve and the one every business case leads with. Interface count determines regression scope, release cadence and, eventually, whether the platform can change at all.

02 **Decide the irreversible things first**

Residency, tenancy and the object model cannot be revisited after the first go-live. Everything else can. Sequence design attention accordingly rather than by what is most visible.

03 **Allocate change capacity by rule**

At eight businesses on one release train, capacity is a contested shared resource. A published allocation rule with a platform reserve is what prevents the fork.

04 **Design for the strictest regulator**

Then relax by exception, and sequence the hardest jurisdiction early. Designing for the easiest and generalising later is the most expensive order available.

None of the four is a technology choice. All four are decided in the first quarter of a programme and determine its cost in year three.

Source: Author's analysis of the reference deployment